

Docker Setup & Running Guide

RL-Based Autonomous Interceptor Drone — training on Ubuntu with Docker, CUDA and an NVIDIA GPU.

```
https://github.com/siddhmehta5131/interceptor-drone-rl
```

This guide takes your project from a brand-new Ubuntu PC to a fully running GPU training job — written for someone who has never used Docker or Ubuntu before. Follow the sections in order.

0. What you are actually setting up

Your project is a **Docker container** on GitHub. A Docker container is a ready-made box that already contains everything the code needs to run (Python, PyTorch with CUDA, Stable-Baselines3, all dependencies). You do **not** install PyTorch or the RL libraries on Ubuntu directly — you install Docker, and Docker runs the box for you.

Three pieces of software are needed on Ubuntu:

Software	What it does
NVIDIA driver	Lets Ubuntu talk to your GPU (the same driver you would use for gaming).
Docker Engine + Compose	Runs containers; the compose file wires everything together.
NVIDIA Container Toolkit	Lets containers <i>see and use</i> the GPU.

How the compose file connects things

- The container image is built from `interceptor-training/Dockerfile` (base: `pytorch/pytorch:2.7.0-cuda12.8-cudnn9-runtime` — CUDA already included).
- Your folders are **mounted** (shared) between Ubuntu and the container:
 - `./data` ↔ `/data` — checkpoints, logs and results are saved here. Anything the container writes to `/data` appears in `interceptor-training/data/` on your PC, so results survive even if the container is deleted.
 - `./configs` ↔ `/data/configs` — the run YAML is shared too, so you can edit settings without rebuilding the image.
- The GPU is passed through, so PyTorch inside the container runs on CUDA.

1. Before you start — checklist

- ☐ A PC with an **NVIDIA GPU** (GTX 10-series or newer; RTX 40/50-series ideal).
- ☐ **Ubuntu 22.04 or 24.04 LTS** installed and booting to the desktop (or a server you log into over SSH).
- ☐ Internet connection.
- ☐ At least **40 GB free disk** (the container image alone is ~10 GB). Check with `df -h`.

No NVIDIA GPU? You can still run on CPU — see section 12.

2. Open a terminal

- On the Ubuntu desktop: press **Ctrl + Alt + T**.
- On a server: connect over SSH from your other PC:

```
ssh siddh@your-server-ip
```

Everything in this guide is typed into the terminal, one command at a time, pressing **Enter** after each. Paste into the terminal with **Ctrl + Shift + V**. Lines starting with `#` are comments — skip them.

3. Update Ubuntu & install basic tools

Update your system (asks for your password; can take a few minutes):

```
# 1. Update the system
sudo apt update
sudo apt upgrade -y
```

Install `git` (to download the project) and `curl` (to download installers), then verify:

```
sudo apt install -y git curl
git --version
curl --version | head -1
```

4. Install the NVIDIA driver

The container provides CUDA **12.8**, which needs an NVIDIA driver of at least **version 570**. The simplest way to install the driver:

```
# 2. NVIDIA driver
sudo ubuntu-drivers autoinstall
```

```
sudo reboot
```

After the PC comes back, open a terminal and check the GPU is visible:

```
nvidia-smi
```

You should see a table listing your GPU (e.g. "NVIDIA GeForce RTX 4070") and a CUDA version in the top-right corner. Anything at driver version 570 or newer is fine, because the container brings its own CUDA 12.8.

No `nvidia-smi` ? The driver didn't install, the GPU is not seated/connected, or it is too old. Try `sudo apt install -y nvidia-driver-550` (or `nvidia-driver-570` if listed), reboot, and re-check.

5. Install Docker Engine + Compose

Use Docker's official one-line installer (installs the engine, the `docker compose` plugin and the builder):

```
# 3. Docker Engine + Compose
curl -fsSL https://get.docker.com | sh
docker --version
docker compose version
```

If `docker compose version` says "command not found":

```
sudo apt update && sudo apt install -y docker-compose-plugin
```

5a. Use Docker without typing "sudo" every time

```
# 4. Docker permission (then log out and back in)
sudo usermod -aG docker $USER
newgrp docker
```

Test it works **without** sudo:

```
docker run hello-world
```

You should see "Hello from Docker!" — that confirms Docker works and your user has permission.

6. Install the NVIDIA Container Toolkit

This is what lets a container use the GPU. Add NVIDIA's software repository and install the toolkit:

```
# 5. GPU-inside-containers support
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | sudo gpg --dearmor -o
/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg
curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.list
| \
  sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg]
https://#g' | \
  sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list > /dev/null
sudo apt update
sudo apt install -y nvidia-container-toolkit
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

Verify the GPU actually reaches containers (pulls a tiny CUDA 12.8 test image):

```
# 6. Verify GPU reaches containers
docker run --rm --gpus all nvidia/cuda:12.8.0-base-ubuntu22.04 nvidia-smi
```

This must print the same GPU table as in section 4. An error like `could not select device driver "nvidia"` means the toolkit isn't configured — re-run the last three commands above and restart Docker.

7. Download your project from GitHub

```
# 7. Get the project
git clone https://github.com/siddhmehta5131/interceptor-drone-rl.git
cd interceptor-drone-rl
ls -la
```

You should see `README.md`, `interceptor-training/`, `src/`, `hover_env.py`, etc. The Docker stuff lives inside `interceptor-training/`:

```
ls interceptor-training
```

Expected: `Dockerfile`, `docker-compose.yml`, `requirements.txt`, `configs/`, `scripts/`, `src/`, `data/`.

8. Recommended first run: the in-container smoke test

Before the long real training, run the built-in **smoke test** — it trains a tiny PPO policy for a few thousand steps across stages 1–3 and checks the whole pipeline (envs, reward, curriculum, checkpointing). If this passes, your GPU + Docker + repo setup is 100% correct.

```
# 8. First run = builds the image (10-30 min download), then runs the checks
cd interceptor-training
```

```
docker compose run --rm interceptor-train --smoke \  
--smoke-config ppo_baseline --smoke-stages 1,2,3 --smoke-steps 1000
```

What happens:

1. First run **builds the image**: downloads the ~9 GB PyTorch base image, then installs your dependencies (torch / SB3 / tensorboard / ...). This is the only slow part — **10–30 minutes** depending on internet speed. Later runs use the cache and start in seconds.
2. The container starts `scripts/train.py --smoke` and runs the checks.
3. On success you'll see a summary with **8 PASS** and "smoke OK".

Lots of download bars and long pauses during the build are normal. If it looks frozen, Docker is still downloading — you can check with `docker images` in a second terminal.

If the smoke test passes, **everything is set up correctly**. Go to section 9.

9. Start the real training

9a. What the default run does

The default config is `configs/default_run.yaml` (mounted at `/data/configs/default_run.yaml` inside the container):

- 4 algorithm presets defined: `ppo_baseline`, `sac_baseline`, `td3_baseline`, `ppo_5layer_deep`. The default is `ppo_baseline`.
- Trains the **8-stage curriculum** (stages 1–7 in the default run list), advancing a stage once its success-rate threshold is met.
- **8 parallel environments**, seed **42**, checkpoint every **50,000 steps**, TensorBoard logging on.
- Each finished stage saves `data/checkpoints/PPO_stage_<N>_final.zip` plus periodic snapshots `PPO_<steps>_steps.zip`; the config-wide final model is saved to `data/results/`.

9b. Run it (foreground)

```
# 9. Start training
docker compose up --build
```

- `--build` rebuilds the image first (fast after the first build — only changed code layers).
- The terminal fills with training logs: per-stage progress, reward components, episode stats, curriculum checks ("advance", "rollback", ...).
- **Stop training:** press **Ctrl + C**. Everything saved so far is safe in `interceptor-training/data/`.

9c. Run it in the background (recommended)

Training can take many hours. Running Docker **detached** keeps it alive even if you close the terminal:

```
docker compose up -d --build
docker compose logs -f
```

(**Ctrl + C** stops *watching* the logs, not the training.) Stop training entirely with `docker compose down`.

10. Monitor with TensorBoard

Metrics are logged to `data/tb_logs/`.

Option A — TensorBoard inside the container:

```
docker compose run --rm --entrypoint tensorboard interceptor-train \
  --logdir /data/tb_logs --host 0.0.0.0 --port 6006
```

Option B — TensorBoard on Ubuntu directly:

```
pip install tensorboard
tensorboard --logdir interceptor-training/data/tb_logs --host 0.0.0.0 --port 6006
```

Then open a browser: `http://localhost:6006` on the same PC, or `http://<ubuntu-ip>:6006` from another PC on the network.

Port already in use? Change `--port 6007` in both places.

11. Evaluate a trained model

Measure a stage's real performance (kill-rate vs. the required threshold):

```
docker compose run --rm --entrypoint python interceptor-train \
  scripts/evaluate.py --config /data/configs/default_run.yaml \
  --config-name ppo_baseline --stage 3 --episodes 50
```

- Auto-picks the best model for stage 3 (stage-final checkpoint, else latest periodic, else the config-wide final model).
- Runs 50 deterministic episodes; prints **kill rate vs. required success rate**, mean reward, mean final distance vs. threshold, and time-to-intercept.
- Saves the summary to `data/results/eval_ppo_baseline_stage_3.json`.

12. Optional: running without a GPU (CPU-only)

1. Edit `interceptor-training/Dockerfile` — change the first line to a CPU image:

```
FROM pytorch/pytorch:2.7.0-cuda12.8-cudnn9-runtime
```

becomes

```
FROM pytorch/pytorch:2.7.0-cpu
```

2. Edit `interceptor-training/docker-compose.yml` — delete `runtime: nvidia`, `NVIDIA_VISIBLE_DEVICES`, `NVIDIA_DRIVER_CAPABILITIES` and the whole `deploy:` block.
3. Rebuild: `docker compose up -d --build`. Training will run on CPU — much slower. Skip sections 4 and 6 entirely.

13. Everyday workflow — quick reference

All commands run from `interceptor-training/`.

Task	Command
First build + start training (foreground)	<code>docker compose up --build</code>
Start training in the background	<code>docker compose up -d --build</code>
Watch training logs	<code>docker compose logs -f</code>
Stop training	<code>docker compose down</code>
Fast sanity check (after any change)	<code>docker compose run --rm interceptor-train --smoke --smoke-config ppo_baseline --smoke-stages 1,2,3 --smoke-steps 1000</code>
Pick a different algorithm preset	Edit <code>default_run.yaml1</code> → <code>config:</code> <code>ppo_5layer_deep</code> (or <code>sac_baseline</code> , <code>td3_baseline</code>)
Change stages / evaluate a stage	Edit <code>default_run.yaml1</code> (<code>stages:</code> list) / <code>scripts/evaluate.py --stage N</code>
Resume an interrupted run	Just run <code>docker compose up -d --build</code> again — the orchestrator detects <code>data/curriculum_state/run_state.json</code> and resumes automatically
Apply new code after <code>git pull</code>	Re-run <code>docker compose up -d --build</code>
Where are my results?	<code>data/checkpoints/</code> (model .zip files) · <code>data/tb_logs/</code> · <code>data/results/</code> · <code>data/curriculum_state/run_state.json</code>
Rebuild from scratch (clean cache)	<code>docker compose build --no-cache</code>

14. Troubleshooting

Symptom	Cause & fix
<code>docker: permission denied ... connect to the docker daemon</code>	User not active in the <code>docker</code> group. Re-run <code>sudo usermod -aG docker \$USER</code> , log out/in, or <code>newgrp docker</code> .
<code>could not select device driver "nvidia"</code>	Toolkit not configured. Re-run <code>sudo nvidia-ctl runtime configure --runtime=docker</code> then <code>sudo systemctl restart docker</code> .
Training runs but is very slow / logs say <code>cpu</code>	GPU not passed through. While training, run <code>docker compose exec interceptor-train nvidia-smi</code> ; re-check section 6.
<code>CUDA out of memory</code>	8 parallel envs too heavy for a small GPU. Edit <code>default_run.yaml</code> → lower <code>n_parallel_envs</code> (e.g. 4), or stop other GPU processes (check with <code>nvidia-smi</code>).
First build takes 30+ minutes	Normal — the CUDA base image is ~9 GB. Don't interrupt it.
<code>docker compose: command not found</code>	Compose plugin missing: <code>sudo apt install -y docker-compose-plugin</code> .
Cannot connect to the Docker daemon	Daemon not running: <code>sudo systemctl start docker</code> (and <code>sudo systemctl enable docker</code> to auto-start on boot).
<code>nvidia-smi: command not found</code> (on Ubuntu)	Driver not installed: <code>sudo ubuntu-drivers autoinstall && sudo reboot</code> .
Disk full / build fails on space	<code>df -h</code> , then <code>docker system prune -f</code> to free space. Your <code>data/</code> folder is safe.
Top-right CUDA in <code>nvidia-smi</code> older than 12.8	Fine — the container ships its own CUDA 12.8. Only the driver version matters (≥ 570).
TensorBoard: <code>port is already allocated</code>	Use another port: <code>--port 6007</code> .
Terminal closed mid-training — progress lost?	No. Checkpoints + <code>run_state.json</code> live in <code>data/</code> . Just start the container again — it resumes.

15. The 10 commands from zero to training

```
# 1. Update the system
sudo apt update && sudo apt upgrade -y
sudo apt install -y git curl

# 2. NVIDIA driver (then REBOOT)
sudo ubuntu-drivers autoinstall
sudo reboot

# 3. Docker Engine + Compose
curl -fsSL https://get.docker.com | sh

# 4. Use Docker without sudo (log out/in after)
sudo usermod -aG docker $USER

# 5. GPU inside containers
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | sudo gpg --dearmor -o
/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg
curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.list
| sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg]
https://#g' | sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list > /dev/null
sudo apt update && sudo apt install -y nvidia-container-toolkit
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker

# 6. Verify GPU reaches containers
docker run --rm --gpus all nvidia/cuda:12.8.0-base-ubuntu22.04 nvidia-smi

# 7. Get the project
git clone https://github.com/siddhmehta5131/interceptor-drone-rl.git
cd interceptor-drone-rl/interceptor-training

# 8. Sanity check (first run builds the image: 10-30 min)
docker compose run --rm interceptor-train --smoke --smoke-config ppo_baseline --smoke-stages
1,2,3 --smoke-steps 1000

# 9. Start training in the background
docker compose up -d --build

# 10. Watch it
docker compose logs -f
```

Backup tip: everything training saves lands in `interceptor-training/data/` on the Ubuntu machine. Copy that folder anywhere (another PC, Google Drive, a USB stick) to back up your trained models.